# 앱 실행 후크 관리

## Astra Control Center

NetApp
April 26, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/astra-control-center-2112/use/execution-hook-examples.html on April 26, 2022. Always check docs.netapp.com for the latest.

# 목차

앱 실행 후크 관리 . . . . . 1
기본 실행 후크 및 정규식 . . . . . 1
사용자 정의 실행 후크에 대한 중요 참고 사항 . . . . . 1
기존 실행 후크를 봅니다 . . . . . 2
사용자 지정 실행 후크를 만듭니다 . . . . . 2
실행 후크를 비활성화합니다 . . . . . 3
실행 후크를 삭제합니다 . . . . . 3
실행 후크 예 . . . . . 4

# 앱 실행 후크 관리

실행 후크는 관리되는 앱의 스냅샷 전후에 실행할 수 있는 사용자 지정 스크립트입니다. 예를 들어 데이터베이스 앱이 있는 경우 실행 후크를 사용하여 스냅샷 전에 모든 데이터베이스 트랜잭션을 일시 중지하고 스냅샷이 완료된 후 트랜잭션을 다시 시작할 수 있습니다. 따라서 애플리케이션 정합성이 보장되는 스냅샷이 보장됩니다.

## 기본 실행 후크 및 정규식

일부 애플리케이션의 경우 Astra Control은 NetApp에서 제공하는 기본 실행 후크와 함께 제공되며, 스냅샷 전후에 고정 및 고정 작업을 처리합니다. Astra Control은 정규식을 사용하여 앱의 컨테이너 이미지를 다음과 같은 앱에 일치시킵니다.

- MariaDB
  - 일치하는 정규식:\bmariadb\b
- MySQL
  - 일치 정규식:\bmysql\b
- PostgreSQL
  - 일치하는 정규식:\bpostgresql\b

일치하는 항목이 있으면 해당 앱에 대한 NetApp 제공 기본 실행 후크가 앱의 활성 실행 후크 목록에 나타나고, 해당 앱의 스냅샷을 생성하면 해당 후크가 자동으로 실행됩니다. 사용자 지정 앱 중 하나에 정규식과 일치하는 유사한 이미지 이름이 있는 경우(기본 실행 후크를 사용하지 않으려는 경우) 이미지 이름을 변경할 수 있습니다. 또는 해당 앱에 대한 기본 실행 후크를 비활성화하고 대신 사용자 지정 후크를 사용합니다.

기본 실행 후크는 삭제하거나 수정할 수 없습니다.

## 사용자 정의 실행 후크에 대한 중요 참고 사항

앱에 대한 실행 후크를 계획할 때 다음 사항을 고려하십시오.

- Astra Control을 사용하려면 실행 가능한 셸 스크립트 형식으로 실행 후크를 작성해야 합니다.
- 스크립트 크기는 128KB로 제한됩니다.
- Astra Control은 실행 후크 설정 및 모든 일치 기준을 사용하여 스냅샷에 적용할 후크를 결정합니다.
- 모든 실행 후크 오류는 소프트 장애이며, 후크에 장애가 발생해도 다른 후크와 스냅샷이 시도됩니다. 그러나 후크가 실패하면 * Activity * 페이지 이벤트 로그에 경고 이벤트가 기록됩니다.
- 실행 후크를 생성, 편집 또는 삭제하려면 소유자, 관리자 또는 구성원 권한이 있는 사용자여야 합니다.
- 실행 후크를 실행하는 데 25분 이상 걸리는 경우 후크에 장애가 발생하고 반환 코드가 "N/A"인 이벤트 로그 항목이 생성됩니다. 영향을 받는 모든 스냅샷은 시간 초과되어 실패로 표시되며, 그 결과 이벤트 로그 항목이 시간 초과를 나타냅니다.

> 실행 후크는 실행 중인 응용 프로그램의 기능을 줄이거나 완전히 비활성화하기 때문에 사용자 지정 실행 후크가 실행되는 시간을 최소화해야 합니다.

스냅샷이 실행되면 실행 후크 이벤트가 다음 순서로 발생합니다.

1. NetApp에서 제공하는 기본 사전 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다.
2. 해당되는 모든 사용자 지정 사전 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다. 필요에 따라 사용자 지정 사전 스냅샷 후크를 생성하고 실행할 수 있지만 스냅샷이 보장되거나 구성 가능해지기 전에 이러한 후크의 실행 순서가 보장되지 않습니다.
3. 스냅샷이 수행됩니다.
4. 해당되는 모든 사용자 지정 사후 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다. 필요에 따라 사용자 지정 사후 스냅샷 후크를 생성하고 실행할 수 있지만 스냅샷 후에 이러한 후크를 실행하는 순서는 보장되거나 구성할 수 없습니다.
5. NetApp에서 제공하는 모든 기본 사후 스냅샷 실행 후크는 해당 컨테이너에서 실행됩니다.

> 운영 환경에서 실행 후크 스크립트를 사용하려면 항상 해당 스크립트를 테스트해야 합니다. 'kubbeck exec' 명령을 사용하여 스크립트를 편리하게 테스트할 수 있습니다. 운영 환경에서 실행 후크를 활성화한 후 결과 스냅샷을 테스트하여 정합성이 보장되는지 확인합니다. 앱을 임시 네임스페이스에 클론 복제하고 스냅샷을 복구한 다음 앱을 테스트하여 이 작업을 수행할 수 있습니다.

# 기존 실행 후크를 봅니다

앱에 대한 기존 맞춤형 또는 NetApp에서 제공한 기본 실행 후크를 볼 수 있습니다.

단계

1. 응용 프로그램 * 으로 이동한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.

   결과 목록에서 사용 가능하거나 비활성화된 실행 후크를 모두 볼 수 있습니다. 후크의 상태, 소스 및 실행 시간(사전 또는 사후 스냅샷)을 확인할 수 있습니다. 실행 후크를 둘러싼 이벤트 로그를 보려면 왼쪽 탐색 영역의 * Activity * 페이지로 이동합니다.

# 사용자 지정 실행 후크를 만듭니다

앱의 사용자 정의 실행 후크를 만들 수 있습니다. 을 참조하십시오 "실행 후크 예" 후크 예 실행 후크를 만들려면 소유자, 관리자 또는 구성원 권한이 있어야 합니다.

> 실행 후크로 사용할 사용자 정의 쉘 스크립트를 작성할 때는 Linux 명령을 실행하거나 실행 파일에 대한 전체 경로를 제공하지 않는 한 파일 시작 부분에 적절한 셸을 지정해야 합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.
3. 새 후크 추가 * 를 선택합니다.
4. 후크 세부 정보 * 영역에서 후크를 실행해야 하는 시기에 따라 * 사전 스냅샷 * 또는 * 사후 스냅샷 * 을 선택합니다.
5. 후크의 고유한 이름을 입력합니다.
6. (선택 사항) 실행 중에 후크에 전달할 인수를 입력하고 각 인수 뒤에 Enter 키를 눌러 각 인수를 기록합니다.

7. Container Images * (컨테이너 이미지 *) 영역에서 응용 프로그램에 포함된 모든 컨테이너 이미지에 대해 후크를 실행해야 하는 경우 * Apply to all container images * (모든 컨테이너 이미지에 적용) 확인란을 활성화합니다. 대신 후크가 하나 이상의 지정된 컨테이너 이미지에만 동작해야 하는 경우 일치시킬 * 컨테이너 이미지 이름 필드에 컨테이너 이미지 이름을 입력합니다.
8. Script * 영역에서 다음 중 하나를 수행합니다.
   - 사용자 지정 스크립트를 업로드합니다.
     i. 파일 업로드 * 옵션을 선택합니다.
     ii. 파일을 찾아 업로드합니다.
     iii. 스크립트에 고유한 이름을 지정합니다.
     iv. (선택 사항) 다른 관리자가 스크립트에 대해 알아야 하는 참고 사항을 입력합니다.
   - 클립보드에서 사용자 정의 스크립트를 붙여 넣습니다.
     i. 클립보드에서 붙여넣기 * 옵션을 선택합니다.
     ii. 텍스트 필드를 선택하고 필드에 스크립트 텍스트를 붙여 넣습니다.
     iii. 스크립트에 고유한 이름을 지정합니다.
     iv. (선택 사항) 다른 관리자가 스크립트에 대해 알아야 하는 참고 사항을 입력합니다.
9. 후크 추가 * 를 선택합니다.

# 실행 후크를 비활성화합니다

앱 스냅샷 전후에 실행 후크가 실행되지 않도록 임시로 설정하려면 실행 후크를 사용하지 않도록 설정할 수 있습니다. 실행 후크를 비활성화하려면 소유자, 관리자 또는 구성원 권한이 있어야 합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.
3. 비활성화하려는 후크에 대해 * 작업 * 드롭다운을 선택합니다.
4. 비활성화 * 를 선택합니다.

# 실행 후크를 삭제합니다

더 이상 필요 없는 경우 실행 후크를 완전히 제거할 수 있습니다. 실행 후크를 삭제하려면 소유자, 관리자 또는 구성원 권한이 있어야 합니다.

단계

1. 응용 프로그램 * 을 선택한 다음 관리되는 응용 프로그램의 이름을 선택합니다.
2. Execution hook * 탭을 선택합니다.
3. 삭제할 후크에 대한 * 작업 * 드롭다운을 선택합니다.
4. 삭제 * 를 선택합니다.

# 실행 후크 예

다음 예제를 사용하여 실행 후크를 구조화하는 방법에 대해 알아보십시오. 이러한 후크를 템플릿 또는 테스트 스크립트로 사용할 수 있습니다.

## 간단한 성공 사례

다음은 표준 출력 및 표준 오류에 성공하여 메시지를 기록하는 간단한 후크의 예입니다.

```
#!/bin/sh

# success_sample.sh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}
```

```
#
# main
#

# log something to stdout
info "running success_sample.sh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

## 단순한 성공 사례(bash 버전)

다음은 bash용으로 작성된 표준 출력 및 표준 오류에 성공하여 메시지를 쓰는 간단한 후크의 예입니다.

```
#!/bin/bash

# success_sample.bash
#
# A simple noop success hook script for testing purposes.
#
# args: None


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}
```

```
#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample.bash"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

### 간단한 성공 사례(zsh 버전)

다음은 Z 셸에 대해 작성된 표준 출력 및 표준 오류에 성공하여 메시지를 기록하는 간단한 후크의 예입니다.

```
#!/bin/zsh

# success_sample.zsh
#
# A simple noop success hook script for testing purposes.
#
# args: None
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}
```

```
#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}

#
# main
#

# log something to stdout
info "running success_sample.zsh"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

## 인수 성공 예제

다음 예제에서는 후크에 args를 사용하는 방법을 보여 줍니다.

```
#!/bin/sh

# success_sample_args.sh
#
# A simple success hook script with args for testing purposes.
#
# args: Up to two optional args that are echoed to stdout

#
# Writes the given message to standard output
#
# $* - The message to write
```

```
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running success_sample_args.sh"


# collect args
arg1=$1
arg2=$2

# output args and arg count to stdout
info "number of args: $#"
info "arg1 ${arg1}"
info "arg2 ${arg2}"

# exit with 0 to indicate success
info "exit 0"
exit 0
```

## 사전 스냅샷/사후 스냅샷 후크의 예

다음 예제에서는 사전 스냅샷 및 사후 스냅샷 후크에 대해 동일한 스크립트를 사용하는 방법을 보여 줍니다.

```
#!/bin/sh

# success_sample_pre_post.sh
#
# A simple success hook script example with an arg for testing purposes
# to demonstrate how the same script can be used for both a prehook and
posthook
#
# args: [pre|post]

# unique error codes for every error case
ebase=100
eusage=$((ebase+1))
ebadstage=$((ebase+2))
epre=$((ebase+3))
epost=$((ebase+4))


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
```

```
error() {
    msg "ERROR: $*" 1>&2
}


#
# Would run prehook steps here
#
prehook() {
    info "Running noop prehook"
    return 0
}

#
# Would run posthook steps here
#
posthook() {
    info "Running noop posthook"
    return 0
}


#
# main
#

# check arg
stage=$1
if [ -z "${stage}" ]; then
    echo "Usage: $0 <pre|post>"
    exit ${eusage}
fi

if [ "${stage}" != "pre" ] && [ "${stage}" != "post" ]; then
    echo "Invalid arg: ${stage}"
    exit ${ebadstage}
fi

# log something to stdout
info "running success_sample_pre_post.sh"

if [ "${stage}" = "pre" ]; then
    prehook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during prehook"
```

```
    fi
fi

if [ "${stage}" = "post" ]; then
    posthook
    rc=$?
    if [ ${rc} -ne 0 ]; then
        error "Error during posthook"
    fi
fi

exit ${rc}
```

## 실패 예

다음 예제에서는 후크의 장애를 처리하는 방법을 보여 줍니다.

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#



#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}



#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}
```

```
#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

## 자세한 정보 표시 실패 예

다음 예제에서는 더 자세한 정보 로깅을 사용하여 후크의 오류를 처리하는 방법을 보여 줍니다.

```
#!/bin/sh

# failure_sample_verbose.sh
#
# A simple failure hook script with args for testing purposes.
#
# args: [The number of lines to output to stdout]


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
```

```
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_sample_verbose.sh"


# output arg value to stdout
linecount=$1
info "line count ${linecount}"

# write out a line to stdout based on line count arg
i=1
while [ "$i" -le ${linecount} ]; do
    info "This is line ${i} from failure_sample_verbose.sh"
    i=$(( i + 1 ))
done

error "exiting with error code 8"
exit 8
```

## 종료 코드 예제에 오류가 발생했습니다

다음 예제에서는 종료 코드와 함께 후크 실패를 보여 줍니다.

```
#!/bin/sh

# failure_sample_arg_exit_code.sh
#
# A simple failure hook script for testing purposes.
#
# args: [the exit code to return]
#


#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#
```

```
# log something to stdout
info "running failure_sample_arg_exit_code.sh"

argexitcode=$1

# log to stderr
error "script failed, returning exit code ${argexitcode}"

# exit with specified exit code
exit ${argexitcode}
```

## 실패 후 성공 예

다음 예제에서는 후크가 처음 실행될 때 후크가 실패하지만 두 번째 실행 후에 후크가 발생하는 방법을 보여 줍니다.

```
#!/bin/sh

# failure_then_success_sample.sh
#
# A hook script that fails on initial run but succeeds on second run for
testing purposes.
#
# Helpful for testing retry logic for post hooks.
#
# args: None
#

#
# Writes the given message to standard output
#
# $* - The message to write
#
msg() {
    echo "$*"
}


#
# Writes the given information message to standard output
#
# $* - The message to write
#
info() {
    msg "INFO: $*"
```

```
}

#
# Writes the given error message to standard error
#
# $* - The message to write
#
error() {
    msg "ERROR: $*" 1>&2
}


#
# main
#

# log something to stdout
info "running failure_success sample.sh"


if [ -e /tmp/hook-test.junk ] ; then
    info "File does exist.  Removing /tmp/hook-test.junk"
    rm /tmp/hook-test.junk
    info "Second run so returning exit code 0"
    exit 0
else
    info "File does not exist.  Creating /tmp/hook-test.junk"
    echo "test" > /tmp/hook-test.junk
    error "Failed first run, returning exit code 5"
    exit 5
fi
```

**Copyright Information**

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

**Trademark Information**

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.